Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE 1108 – Compiladores e Intérpretes

Proyecto: Compilador para Código Binario

Especificación de la Arquitectura y Documentación del Compilador

Estudiante	Carné
Camila Lizano Brenes	2024255324
Fabiana Moreno Castrillo	2023153700
Felipe Viales Vieto	2021030084
Daniel González Chaves	2022279667

Profesor: Jason Leitón Jiménez

Fecha de entrega: 14 de octubre de 2026, 13:00

Repositorio del proyecto: https://github.com/FabiMC040105/CompiladoresIIS2026.git

Índice

3. Especificación de la Arquitectura **2**

3.1. Tamaño de palabra 2

3.2. Conjunto de registros 2

3.3. Organización de memoria 2

3.4. Conjunto de instrucciones 3

8. Documentación del Compilador **6**

8.1. Especificación de la Arquitectura 6

8.2. Especificación del Lenguaje de Entrada 6

8.3. Descripción de Algoritmos Clave 7

8.4. Ejemplos de Ejecución 9

8.5. Manejo de Casos Especiales 66

3. Especificación de la Arquitectura

La arquitectura objetivo corresponde al procesador VLIW desarrollado por el grupo en Arquitectura de Computadores I. El compilador genera ensamblador para esta ISA y debe respetar tanto su codificación como las restricciones de ejecución del pipeline.

3.1. Tamaño de palabra

Los datos se manejan en palabras de 32 bits. Cada instrucción VLIW completa, denominada *bundle*, tiene 128 bits y contiene cuatro slots fijos de 32 bits:

FEIST [127:96]	**BRU [95:64]**	**LSU [63:32]**	**ALU [31:0]**
32 bits	32 bits	32 bits	32 bits

El slot ALU ocupa los bits menos significativos y FEIST los más significativos. Los campos no utilizados se rellenan con cero. Un bundle ocupa 16 bytes, por lo que el PC avanza 16 bytes en ejecución secuencial. El orden de bytes es *little-endian*.

3.2. Conjunto de registros

La arquitectura dispone de 16 registros identificados de `x0` a `x15`. El número del registro es también su código de 4 bits dentro de las instrucciones.

Cuadro 1: Registros y convención utilizada por el compilador.

Registro	Propósito
`x0`	Constante cero; las escrituras se descartan.
`x1`	Program Counter (PC). Leerlo entrega el PC del bundle; una escritura válida redirige el flujo.
`x2`	Valor de retorno de funciones por convenio del backend.
`x3`	Registro auxiliar reservado por el backend.
`x4-x12`	Registros temporales.
`x13`	Registro auxiliar para direcciones y cálculos de memoria.
`x14`	Stack Pointer (SP).
`x15`	Frame Pointer (FP).

3.3. Organización de memoria

La memoria de datos implementada es de 64 KiB. El compilador la divide lógicamente de la siguiente forma:

Cuadro 2: Organización lógica de la memoria de datos.

Región	**Rango**	**Uso**
Datos globales	`0x0000-0x7FFF`	Variables globales
Pila	`0x8000-0xFFFF`	Parámetros, marcos y variables locales

Los escalares `entero` y `vof` ocupan 4 bytes. Los elementos de listas y matrices también ocupan 4 bytes. Las variables globales se asignan desde direcciones bajas hacia arriba, mientras que la pila crece hacia direcciones menores. El SP se inicializa conceptualmente en `0x10000`; la primera reserva lo desplaza a una dirección válida dentro de la región de pila. Los accesos `LOAD` y `STORE` son de 32 bits y se realizan sobre direcciones alineadas a 4 bytes.

La memoria de instrucciones está separada de la memoria de datos. Cada bundle consume 16 bytes; con una memoria de instrucciones de 64 KiB pueden almacenarse hasta 4096 bundles. La versión actual no utiliza *heap* ni asignación dinámica.

3.4. Conjunto de instrucciones

Cuadro 3: Tabla completa de instrucciones de la ISA.

Instr.	**Slot**	**Formato**	**Sintaxis**	**Opcode**	**Descripción**
NOP	Todos	–	`NOP`	0000	No realiza operación; el slot se representa con ceros.
ADD	ALU	R	`ADD rd, rs1, rs2`	0001	Suma de 32 bits.
SUB	ALU	R	`SUB rd, rs1, rs2`	0010	Resta de 32 bits.
OR	ALU	R	`OR rd, rs1, rs2`	0011	OR bit a bit.
XOR	ALU	R	`XOR rd, rs1, rs2`	0100	XOR bit a bit.
AND	ALU	R	`AND rd, rs1, rs2`	0101	AND bit a bit.
SLT	ALU	R	`SLT rd, rs1, rs2`	0110	Escribe 1 si `rs1 <rs2` con signo.
ADDI	ALU	I	`ADDI rd, rs1, imm`	1001	Suma un inmediato con signo de 12 bits.
SRLI	ALU	I	`SRLI rd, rs1, imm`	1010	Desplazamiento lógico a la derecha.
SLLI	ALU	I	`SLLI rd, rs1, imm`	1011	Desplazamiento lógico a la izquierda.
LOAD	LSU	L	`LOAD rd, off(rb)`	01	Carga una palabra de memoria.
STORE	LSU	S	`STORE rs, off(rb)`	10	Almacena una palabra en memoria.
JMP	BRU	J	`JMP offset`	001	Salto incondicional PC-relativo.
BGE	BRU	B	`BGE rs1, rs2, offset`	010	Salta si `rs1 >= rs2`.
BLE	BRU	B	`BLE rs1, rs2, offset`	011	Salta si `rs1 <= rs2`.
BEQ	BRU	B	`BEQ rs1, rs2, offset`	100	Salta si los operandos son iguales.
BNE	BRU	B	`BNE rs1, rs2, offset`	101	Salta si los operandos son diferentes.
AUTH	FEIST	FA	`AUTH rs1, rs2`	001	Intenta autorizar el acceso a la bóveda.
LOGOUT	FEIST	FL	`LOGOUT`	010	Elimina la autorización actual.
KSTORE	FEIST	FK	`KSTORE key, round, rs1`	011	Guarda una subllave en la bóveda.
FROUND	FEIST	FR	`FROUND rdL, rdR, rsL, rsR, key, round, mode`	100	Ejecuta una ronda de Feistel4.

Ejemplos de uso:

```
ALU: ADD x4, x2, x3
ALU: ADDI x5, x4, 12
LSU: LOAD x6, -4(x15)
LSU: STORE x6, -8(x15)
BRU: BNE x4, x0, 64
BRU: JMP -160
FEIST: AUTH x2, x3
FEIST: KSTORE 0, 2, x4
FEIST: FROUND x7, x8, x5, x6, 0, 2, 0
FEIST: LOGOUT
```

Las operaciones de alto nivel `MUL`, `DIV`, módulo, potencia, llamadas y retornos no tienen un opcode nativo propio; el compilador las sintetiza mediante secuencias de instrucciones disponibles.

3.4.1. Formatos principales

ALU tipo R:

$$[31{:}16] = 0, \quad [15{:}12] = rs2, \quad [11{:}8] = rs1, \quad [7{:}4] = rd, \quad [3] = 0, \quad [2{:}0] = opcode$$

ALU tipo I:

$$[31{:}24] = 0, \quad [23{:}12] = imm, \quad [11{:}8] = rs1, \quad [7{:}4] = rd, \quad [3] = 1, \quad [2{:}0] = opcode$$

LSU:

$$[31{:}22] = 0, \quad [21{:}10] = offset, \quad [9{:}6] = rb, \quad [5{:}2] = reg, \quad [1{:}0] = opcode$$

BRU condicional:

$$[31{:}23] = 0, \quad [22{:}11] = offset, \quad [10{:}7] = rs2, \quad [6{:}3] = rs1, \quad [2{:}0] = opcode$$

JMP:

$$[31{:}23] = 0, \quad [22{:}3] = offset, \quad [2{:}0] = 001$$

Los offsets de BRU se expresan en bytes y se suman al PC del bundle que contiene el salto. Los condicionales utilizan 12 bits con signo y `JMP` utiliza 20 bits con signo.

8. Documentación del Compilador

El código fuente, las pruebas y los artefactos generados del proyecto se encuentran en el repositorio oficial del grupo: https://github.com/FabiMC040105/CompiladoresIIS2026.git.

8.1. Especificación de la Arquitectura

La especificación solicitada en este apartado corresponde a la misma arquitectura objetivo documentada en la Sección 3. Allí se incluyen el tamaño de palabra e instrucción, registros y propósitos, organización de memoria, tabla completa del ISA, opcodes, formatos y ejemplos de uso.

8.2. Especificación del Lenguaje de Entrada

El lenguaje fuente utiliza la extensión `.e`. Las sentencias terminan con la secuencia `:)` y los bloques se delimitan con llaves.

8.2.1. Tokens

Cuadro 4: Resumen de tokens del lenguaje.

Categoría	Elementos reconocidos
Palabras reservadas	`si`, `sinosi`, `sino`, `mientras`, `para`, `defina`, `devuelva`, `rompa`, `principal`.
Tipos	`entero`, `vof`, `vacio`.
Colecciones	`lista`, `matriz`.
Booleanos	`verdadero`, `falso`.
Aritméticos	`+`, `-`, `*`, `/`, `%`, `**`.
Comparación	`==`, `!=`, `<`, `>`, `<=`, `>=`.
Lógicos	`$$` (AND) y `-` (OR).
Delimitadores	`{ } ( ) [ ] ,` y terminador `:)`.
Identificadores	Deben iniciar con letra minúscula; luego pueden contener letras, dígitos o `_`.
Números	`0` o enteros positivos sin ceros iniciales; el signo negativo se maneja como operador.

Los comentarios de una línea inician con el carácter exttt^. Los comentarios de bloque se delimitan con dos caracteres exttt^^.

```
^ comentario de una linea

^^
comentario de varias lineas
^^
```

8.2.2. Variables, asignaciones y expresiones

```
entero contador = 0 :)
vof activo = verdadero :)
contador = contador + 1 :)
activo = contador < 10 :)
```

Las listas y matrices son estáticas. Cada elemento ocupa 4 bytes y las matrices se almacenan por filas (*row-major*):

```
entero lista numeros[4] = [10, 20, 30, 40] :)
entero matriz tabla[2][2] = [[1, 2], [3, 4]] :)
tabla[1][0] = numeros[2] :)
```

Para una matriz con C columnas:

$$\text{dir}(i, j) = \text{base} + 4(iC + j)$$

8.2.3. Estructuras de control y funciones

```
si (x > 10) {
    x = x - 1 :)
} sinosi (x == 10) {
    x = x + 2 :)
} sino {
    x = x + 1 :)
} :)

mientras (i < 10) {
    i = i + 1 :)
} :)

para (entero j = 0, j < 5, j = j + 1) {
    x = x + j :)
} :)

defina entero sumar(entero a, entero b) {
    devuelva a + b :)
} :)
```

La sentencia `rompa :)` termina el ciclo más interno. Las funciones pueden recibir parámetros y devolver valores mediante `devuelva`. El compilador controla ámbitos globales, de función y de bloque; por ello, dos identificadores con el mismo nombre pueden representar símbolos distintos si pertenecen a ámbitos diferentes.

8.3. Descripción de Algoritmos Clave

8.3.1. Análisis léxico

El archivo `lexer.l` implementa el analizador con Flex. Cada regla asocia un patrón con un token de Bison. Los espacios y comentarios se ignoran; los lexemas válidos retornan su token y los caracteres

no reconocidos producen error léxico. Para comentarios de bloque se utiliza un estado exclusivo, lo cual evita que su contenido sea interpretado como código.

8.3.2. Análisis sintáctico y AST

El archivo `parser.y` define la gramática en Bison. Cuando una producción es reconocida se construyen nodos del árbol de sintaxis abstracta (AST). Cada nodo conserva su tipo, valor cuando corresponde, línea de origen, primer hijo y siguiente hermano. Esto permite representar bloques y secuencias de instrucciones sin almacenar detalles sintácticos innecesarios.

8.3.3. Tabla de símbolos y memoria

La tabla de símbolos registra nombre, categoría, tipo, forma, ámbito y ubicación de memoria. La búsqueda de identificadores comienza en el ámbito actual y continúa por sus ámbitos padre. Las variables globales reciben direcciones absolutas; los parámetros y locales utilizan desplazamientos respecto a `x15` (FP).

La convención principal del marco de función es:

Posición	**Contenido**
`FP+8, FP+12, ...`	Parámetros
`FP+4`	Dirección de retorno
`FP+0`	FP anterior
`FP-4, FP-8, ...`	Variables locales

8.3.4. Generación de ensamblador y riesgos de datos

El backend recorre el AST y emite instrucciones ALU, LSU, BRU y FEIST. Como el procesador usa el pipeline

$$\texttt{IF} \rightarrow \texttt{ID} \rightarrow \texttt{RR} \rightarrow \texttt{EX} \rightarrow \texttt{WB},$$

el compilador calendariza las dependencias RAW. Si un productor se emite en el bundle B, el valor puede consumirse de forma segura a partir de $B + 3$. Por ello, una dependencia inmediata requiere 2 NOPs; si ya existe un bundle independiente requiere 1; y con dos bundles independientes no requiere NOP adicional.

```
ALU: ADDI x4, x0, 5
ALU: NOP
ALU: NOP
ALU: ADD x5, x4, x6
```

Las escrituras a `x1` se tratan de forma especial porque redirigen el PC. El backend agrega una barrera de control de un bundle después del retorno por software.

8.3.5. Cálculo de saltos y resolución de etiquetas

Las estructuras de control se generan inicialmente con etiquetas simbólicas. El resolvedor realiza dos pasos:

1. recorre el ensamblador y registra cada etiqueta junto con el índice de bundle en el que se encuentra;
2. recorre las referencias, localiza su destino y sustituye el nombre por un desplazamiento numérico.

Como cada bundle ocupa 16 bytes, para una referencia desde el bundle B_o hacia B_d se calcula:

$$\text{offset} = (B_d - B_o) \times 16$$

La resolución se hace después de insertar los NOPs, ya que estos cambian la posición definitiva de los saltos. El módulo también detecta etiquetas duplicadas, referencias inexistentes y desplazamientos fuera del rango representable por la instrucción.

8.3.6. Resolución de referencias a funciones

Las llamadas y retornos se implementan por software. El llamador preserva temporales, coloca argumentos en la pila y salta a la etiqueta de la función. El valor retornado se recibe en `x2`. La dirección de retorno se maneja simbólicamente durante la generación y se convierte posteriormente a un desplazamiento respecto al PC. El callee restaura el marco anterior y escribe la dirección de retorno en `x1` para redirigir la ejecución al llamador.

8.4. Ejemplos de Ejecución

Para validar el flujo completo del compilador se utilizaron los tres programas requeridos por el proyecto. En cada caso se conserva el código fuente, el ensamblador generado, la representación hexadecimal de los bundles VLIW y una traza de ejecución obtenida sobre el procesador de Arquitectura de Computadores I. Los archivos de evidencia se almacenan en `docs/evidencias/`.

Para mantener el documento legible, la salida verbosa completa del compilador se conserva como archivo de evidencia en el repositorio, mientras que en esta sección se presenta un resumen de las fases y cantidades generadas. El código fuente, el ensamblador, el HEX y las trazas de ejecución sí se incluyen directamente.

La representación hexadecimal contiene una línea de 32 dígitos hexadecimales por bundle, equivalentes a 128 bits. Para la validación funcional se cargó cada archivo en la memoria de instrucciones del procesador y se ejecutó con el banco de pruebas de integración `tb_compilador.sv`. La llegada al bucle terminal `fin_programa`, junto con los valores finales de registros y memoria, se utilizó como criterio de éxito.

Cuadro 5: Resumen de resultados de los tres programas de validación.

Programa	Resultado validado	PC de `fin_programa`	Estado
Factorial	$5! = 120$; el valor final se observa en `x2` y en memoria.	`0x00000F70`	PASS
Búsqueda en matriz	Para el objetivo 23 se obtiene `posicion=4`, `fila=1` y `columna=1`.	`0x00001950`	PASS
MCD	$\text{MCD}(48, 18) = 6$; el valor retornado queda en `x2`.	`0x00002000`	PASS

8.4.1. Ejercicio 1: cálculo de factorial

El primer programa calcula el factorial de 5 mediante una función, variables locales, un condicional y un ciclo. La ejecución completa produjo el valor 120 y alcanzó correctamente `fin_programa` en `0x00000F70`.

Código fuente completo.

```
defina entero factorial(entero n) {
    entero resultado = 1 :)
    entero i = 1 :)

    si (n == 0) {
        devuelva 1 :)
    } :)

    mientras (i <= n) {
        resultado = resultado * i :)
        i = i + 1 :)
    } :)

    devuelva resultado :)
} :)

defina vacio principal() {
    entero n = 5 :)
    entero resultado = factorial(n) :)
} :)
```

Resumen de compilación.

```
[FASES 1-2] Analisis lexico, sintactico y construccion del AST...
Programa sintacticamente correcto.
[FASE 3] Analisis semantico finalizado sin errores.
[FASE 4] Generando ensamblador y calendarizando dependencias...
[FASE 5] Resolviendo etiquetas y offsets...
Fase 5: 249 bundles resueltos correctamente
[FASE 5.5] Empaquetando operaciones independientes en bundles VLIW...
Empaquetado VLIW: 249 bundles conservados
  ALU + LSU: 3
  ALU + BRU: 1
  LSU + BRU: 2
[FASE 6] Generando codigo maquina...
```

```
Archivo binario generado: factorial_vliw.bin
Bundles generados: 249
Bytes de instrucciones: 3984
```

Ensamblador intermedio generado.

```
ALU: ADDI x14, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 1
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: ADD x15, x14, x0
BRU: JMP 1296
func_factorial:
ALU: ADDI x2, x0, 0
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x15, 0(x14)
ALU: ADD x15, x14, x0
ALU: ADDI x14, x14, -8
ALU: ADDI x4, x0, 1
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
ALU: ADDI x4, x0, 1
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15)
ALU: ADDI x5, x0, 0 | LSU: LOAD x4, 8(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BNE x4, x5, 96
ALU: ADDI x4, x0, 1
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 784
```

```
fin_if_0:
inicio_while_1:
LSU: LOAD x4, -8(x15)
LSU: LOAD x5, 8(x15)
ALU: NOP
ALU: NOP
ALU: SLT x4, x5, x4
ALU: NOP
ALU: NOP
BRU: BNE x4, x0, 576
LSU: LOAD x4, -4(x15)
ALU: ADDI x6, x0, 0 | LSU: LOAD x5, -8(x15)
ALU: NOP
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 1 | BRU: BGE x4, x0, 64
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x7, x7, 1
mul_izq_positivo_3:
BRU: BGE x5, x0, 48
ALU: SUB x5, x0, x5
ALU: ADDI x7, x7, 1
mul_der_positivo_4:
mul_bucle_5:
ALU: NOP
BRU: BEQ x5, x0, 144
ALU: AND x9, x5, x8
ALU: NOP
ALU: NOP
BRU: BEQ x9, x0, 32
ALU: ADD x6, x6, x4
mul_sin_suma_6:
ALU: SLLI x4, x4, 1
ALU: SRLI x5, x5, 1
BRU: JMP -144
mul_fin_bucle_7:
BRU: BNE x7, x8, 32
ALU: SUB x6, x0, x6
mul_fin_signo_8:
ALU: NOP
ALU: NOP
LSU: STORE x6, -4(x15)
ALU: ADDI x5, x0, 1 | LSU: LOAD x4, -8(x15)
ALU: NOP
ALU: NOP
ALU: NOP
ALU: ADD x4, x4, x5
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15) | BRU: JMP -656
ALU: NOP
fin_while_2:
LSU: LOAD x4, -4(x15)
ALU: NOP
```

```
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 16
ret_factorial:
ALU: ADD x14, x15, x0
LSU: LOAD x3, 0(x15)
LSU: LOAD x13, 4(x15)
ALU: ADDI x14, x15, 8
ALU: ADD x15, x3, x0
ALU: ADD x1, x13, x0
ALU: NOP
principal:
ALU: ADD x15, x14, x0
ALU: ADDI x13, x14, -8
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 1936
pila_ok_9:
ALU: ADDI x14, x14, -8
ALU: ADDI x4, x0, 5
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
ALU: ADDI x13, x14, -56
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
```

```
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 1472
pila_ok_10:
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x5, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x6, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x7, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x8, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x9, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x10, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x11, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x12, 0(x14)
LSU: LOAD x4, -4(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
```

```
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: ADDI x3, x1, 80
ALU: NOP
ALU: NOP
LSU: STORE x3, 0(x14) | BRU: JMP -2864
ALU: NOP
ret_call_11:
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x12, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x11, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x10, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x9, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x8, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x7, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x6, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x5, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x14)
ALU: ADDI x14, x14, 4
ALU: ADD x4, x0, x2
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15)
ALU: ADD x14, x15, x0
BRU: JMP 16
fin_programa:
BRU: JMP 0
error_pila:
```

```
BRU: JMP 0
```

Representación hexadecimal legible.

```
000000000000000000000000000000E9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000001EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000EF1
000000000000288100000000000000000
00000000000000000000000000000029
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000003BE00000000
000000000000000000000000000000EF1
0000000000000000000000000FF8EE9
00000000000000000000000000001049
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000003FF3D200000000
00000000000000000000000000001049
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000003FE3D200000000
00000000000000000023D100000059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000302A50000000000000000
00000000000000000000000000001049
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004021
00000000000018810000000000000000
00000000000000003FE3D100000000
00000000000000000023D500000000
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000000000004546
00000000000000000000000000000000
00000000000000000000000000000000
00000000012002500000000000000000
00000000000000003FF3D100000000
00000000000000003FE3D500000069
00000000000000000000000000000000
00000000000000000000000000000079
00000000002002200000000000001089
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000001779
00000000001802A0000000000000000
00000000000000000000000000005052
00000000000000000000000000001779
00000000000000000000000000000000
00000000004802C0000000000000000
00000000000000000000000000008595
00000000000000000000000000000000
00000000000000000000000000000000
00000000001004C0000000000000000
00000000000000000000000000004661
00000000000000000000000000000144B
00000000000000000000000000000155A
00000000007FFB810000000000000000
00000000001043D0000000000000000
00000000000000000000000000006062
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000003FF3DA00000000
00000000000000003FE3D100001059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000005441
00000000000000000000000000000000
00000000000000000000000000000000
00000000007FEB81003FE3D200000000
00000000000000000000000000000000
00000000000000003FF3D100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004021
00000000000000810000000000000000
00000000000000000000000000000FE1
00000000000000000003CD00000000
00000000000000000013F500000000
00000000000000000000000000008FE9
00000000000000000000000000003F1
00000000000000000000000000000D11
00000000000000000000000000000000
00000000000000000000000000000EF1
00000000000000000000000000FF8ED9
```

```
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
0000000000003C810000000000000000
000000000000000000000000000FF8EE9
00000000000000000000000000005049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FF3D200000000
000000000000000000000000000FC8ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
0000000000002E010000000000000000
000000000000000000000000000FFCEE9
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000039200000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039A00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039E00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A200000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AA00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AE00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B200000000
00000000000000000003FF3D100000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
0000000000000000000000000FFCEE9
00000000000000000000000000050139
00000000000000000000000000000000
00000000000000000000000000000000
00000000007FA6810000038E00000000
00000000000000000000000000000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AD00000000
00000000000000000000000000004EE9
```

```
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A500000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039D00000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039500000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039100000000
00000000000000000000000000004EE9
00000000000000000000000000002041
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FE3D200000000
00000000000000000000000000000FE1
00000000000000081000000000000000
00000000000000001000000000000000
00000000000000001000000000000000
```

Traza de ejecución. En la traza se observa la evolución del resultado parcial hasta 120, la restauración del marco de pila y la detección del bucle terminal. En particular, `x2=120` corresponde al valor retornado por la función factorial.

```
BRANCH PC=00000160 -> 00000670
BRANCH PC=000007e0 -> 00000800
STORE MEM[0000fffc] <= 00000005 (5)
BRANCH PC=000009b0 -> 000009d0
STORE MEM[0000fff4] <= 00000005 (5)
STORE MEM[0000fff0] <= 00000000 (0)
STORE MEM[0000ffec] <= 00000000 (0)
STORE MEM[0000ffe8] <= 00000000 (0)
STORE MEM[0000ffe4] <= 00000000 (0)
STORE MEM[0000ffe0] <= 00000000 (0)
STORE MEM[0000ffdc] <= 00000000 (0)
STORE MEM[0000ffd8] <= 00000000 (0)
STORE MEM[0000ffd4] <= 00000000 (0)
STORE MEM[0000ffd0] <= 00000005 (5)
```

```
  STORE MEM[0000ffcc] <= 00000cc0 (3264)
  BRANCH PC=00000ca0 -> 00000170
  STORE MEM[0000ffc8] <= 00010000 (65536)
  STORE MEM[0000ffc4] <= 00000001 (1)
  STORE MEM[0000ffc0] <= 00000001 (1)
  BRANCH PC=000002a0 -> 00000300
  BRANCH PC=000003c0 -> 00000400
  BRANCH PC=00000400 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000440 -> 000004d0
  BRANCH PC=000004d0 -> 000004f0
  STORE MEM[0000ffc4] <= 00000001 (1)
  STORE MEM[0000ffc0] <= 00000002 (2)
  BRANCH PC=00000590 -> 00000300
  BRANCH PC=000003c0 -> 00000400
  BRANCH PC=00000400 -> 00000430
  BRANCH PC=00000480 -> 000004a0
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000440 -> 000004d0
  BRANCH PC=000004d0 -> 000004f0
  STORE MEM[0000ffc4] <= 00000002 (2)
  STORE MEM[0000ffc0] <= 00000003 (3)
  BRANCH PC=00000590 -> 00000300
  BRANCH PC=000003c0 -> 00000400
  BRANCH PC=00000400 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000440 -> 000004d0
  BRANCH PC=000004d0 -> 000004f0
  STORE MEM[0000ffc4] <= 00000006 (6)
  STORE MEM[0000ffc0] <= 00000004 (4)
  BRANCH PC=00000590 -> 00000300
  BRANCH PC=000003c0 -> 00000400
  BRANCH PC=00000400 -> 00000430
  BRANCH PC=00000480 -> 000004a0
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000480 -> 000004a0
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000440 -> 000004d0
  BRANCH PC=000004d0 -> 000004f0
  STORE MEM[0000ffc4] <= 00000018 (24)
  STORE MEM[0000ffc0] <= 00000005 (5)
  BRANCH PC=00000590 -> 00000300
  BRANCH PC=000003c0 -> 00000400
  BRANCH PC=00000400 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000480 -> 000004a0
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=000004c0 -> 00000430
  BRANCH PC=00000440 -> 000004d0
  BRANCH PC=000004d0 -> 000004f0
  STORE MEM[0000ffc4] <= 00000078 (120)
  STORE MEM[0000ffc0] <= 00000006 (6)
  BRANCH PC=00000590 -> 00000300
  BRANCH PC=00000370 -> 000005b0
  BRANCH PC=000005f0 -> 00000600
  STORE MEM[0000fff8] <= 00000078 (120)
  BRANCH PC=00000f60 -> 00000f70
  BRANCH PC=00000f70 -> 00000f70
SELF_LOOP detectado en PC=0x00000f70
PASS: fin_programa alcanzado en 0x00000f70
```

```
x0 = 0x00000000 (0)
x1 = 0x00000000 (0)
x2 = 0x00000078 (120)
x3 = 0x00010000 (65536)
x4 = 0x00000078 (120)
x5 = 0x00000000 (0)
x6 = 0x00000000 (0)
x7 = 0x00000000 (0)
x8 = 0x00000000 (0)
x9 = 0x00000000 (0)
x10 = 0x00000000 (0)
x11 = 0x00000000 (0)
x12 = 0x00000000 (0)
x13 = 0x00000cc0 (3264)
x14 = 0x00010000 (65536)
x15 = 0x00010000 (65536)
MEM[0000ffc0] = 00000006 (6)
MEM[0000ffc4] = 00000078 (120)
MEM[0000ffc8] = 00010000 (65536)
MEM[0000ffcc] = 00000cc0 (3264)
MEM[0000ffd0] = 00000005 (5)
MEM[0000fff4] = 00000005 (5)
MEM[0000fff8] = 00000078 (120)
MEM[0000fffc] = 00000005 (5)
```

8.4.2. Ejercicio 2: búsqueda en matriz bidimensional

El segundo programa declara la matriz

$$\begin{bmatrix} 4 & 8 & 15 \\ 16 & 23 & 42 \end{bmatrix}$$

y busca el valor 23 mediante dos ciclos anidados. La función devuelve la posición lineal en orden por filas. El resultado validado fue `posicion=4`; posteriormente el programa calcula `fila=1` y `columna=1`. El procesador alcanzó `fin_programa` en `0x00001950`.

Código fuente completo.

```
defina entero buscar(entero matriz datos[2][3], entero objetivo) {
    entero i = 0 :)
    entero j = 0 :)

    mientras (i < 2) {
        j = 0 :)

        mientras (j < 3) {
            si (datos[i][j] == objetivo) {
                devuelva i * 3 + j :)
            } :)

            j = j + 1 :)
        } :)

        i = i + 1 :)
    } :)

    devuelva -1 :)
} :)

defina vacio principal() {
    entero matriz datos[2][3] = [[4, 8, 15], [16, 23, 42]] :)
    entero objetivo = 23 :)

    entero posicion = buscar(datos, objetivo) :)
    entero fila = posicion / 3 :)
    entero columna = posicion % 3 :)
} :)
```

Resumen de compilación.

```
[FASES 1-2] Analisis lexico, sintactico y construccion del AST...
Programa sintacticamente correcto.
[FASE 3] Analisis semantico finalizado sin errores.
[FASE 4] Generando ensamblador y calendarizando dependencias...
[FASE 5] Resolviendo etiquetas y offsets...
Fase 5: 407 bundles resueltos correctamente
[FASE 5.5] Empaquetando operaciones independientes en bundles VLIW...
Empaquetado VLIW: 407 bundles conservados
  ALU + LSU: 8
  ALU + BRU: 4
```

```
  LSU + BRU: 3
[FASE 6] Generando codigo maquina...
Archivo binario generado: ejercicio2_matriz.bin
Bundles generados: 407
Bytes de instrucciones: 6512
```

Ensamblador intermedio generado.

```
ALU: ADDI x14, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 1
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: ADD x15, x14, x0
BRU: JMP 2160
func_buscar:
ALU: ADDI x2, x0, 0
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x15, 0(x14)
ALU: ADD x15, x14, x0
ALU: ADDI x14, x14, -8
ALU: ADDI x4, x0, 0
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
ALU: ADDI x4, x0, 0
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15)
inicio_while_0:
ALU: ADDI x5, x0, 2 | LSU: LOAD x4, -4(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BGE x4, x5, 1600
ALU: ADDI x4, x0, 0
ALU: NOP
```

```
ALU: NOP
LSU: STORE x4, -8(x15)
inicio_while_2:
ALU: ADDI x5, x0, 3 | LSU: LOAD x4, -8(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BGE x4, x5, 1312
LSU: LOAD x4, -4(x15)
ALU: ADDI x6, x0, 3 | LSU: LOAD x5, -8(x15)
ALU: NOP
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 0
ALU: ADDI x9, x0, 1 | BRU: BGE x4, x0, 64
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x8, x8, 1
mul_izq_positivo_5:
BRU: BGE x6, x0, 48
ALU: SUB x6, x0, x6
ALU: ADDI x8, x8, 1
mul_der_positivo_6:
mul_bucle_7:
ALU: NOP
BRU: BEQ x6, x0, 144
ALU: AND x10, x6, x9
ALU: NOP
ALU: NOP
BRU: BEQ x10, x0, 32
ALU: ADD x7, x7, x4
mul_sin_suma_8:
ALU: SLLI x4, x4, 1
ALU: SRLI x6, x6, 1
BRU: JMP -144
mul_fin_bucle_9:
BRU: BNE x8, x9, 32
ALU: SUB x7, x0, x7
mul_fin_signo_10:
ALU: NOP
ALU: NOP
ALU: ADD x7, x7, x5
ALU: NOP
ALU: NOP
ALU: SLLI x7, x7, 2
LSU: LOAD x13, 8(x15)
ALU: NOP
ALU: NOP
ALU: ADD x13, x13, x7
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x13)
LSU: LOAD x5, 12(x15)
ALU: NOP
ALU: NOP
```

```
BRU: BNE x4, x5, 512
ALU: ADDI x5, x0, 3 | LSU: LOAD x4, -4(x15)
ALU: NOP
ALU: ADDI x6, x0, 0
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 1 | BRU: BGE x4, x0, 64
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x7, x7, 1
mul_izq_positivo_11:
BRU: BGE x5, x0, 48
ALU: SUB x5, x0, x5
ALU: ADDI x7, x7, 1
mul_der_positivo_12:
mul_bucle_13:
ALU: NOP
BRU: BEQ x5, x0, 144
ALU: AND x9, x5, x8
ALU: NOP
ALU: NOP
BRU: BEQ x9, x0, 32
ALU: ADD x6, x6, x4
mul_sin_suma_14:
ALU: SLLI x4, x4, 1
ALU: SRLI x5, x5, 1
BRU: JMP -144
mul_fin_bucle_15:
BRU: BNE x7, x8, 32
ALU: SUB x6, x0, x6
mul_fin_signo_16:
LSU: LOAD x4, -8(x15)
ALU: NOP
ALU: NOP
ALU: ADD x6, x6, x4
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x6
BRU: JMP 432
fin_if_4:
ALU: ADDI x5, x0, 1 | LSU: LOAD x4, -8(x15)
ALU: NOP
ALU: NOP
ALU: NOP
ALU: ADD x4, x4, x5
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15) | BRU: JMP -1344
ALU: NOP
fin_while_3:
ALU: ADDI x5, x0, 1 | LSU: LOAD x4, -4(x15)
ALU: NOP
ALU: NOP
ALU: NOP
ALU: ADD x4, x4, x5
```

```
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15) | BRU: JMP -1632
ALU: NOP
fin_while_1:
ALU: ADDI x4, x0, 1
ALU: NOP
ALU: NOP
ALU: SUB x4, x0, x4
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 16
ret_buscar:
ALU: ADD x14, x15, x0
LSU: LOAD x3, 0(x15)
LSU: LOAD x13, 4(x15)
ALU: ADDI x14, x15, 8
ALU: ADD x15, x3, x0
ALU: ADD x1, x13, x0
ALU: NOP
principal:
ALU: ADD x15, x14, x0
ALU: ADDI x13, x14, -40
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 3600
pila_ok_17:
ALU: ADDI x14, x14, -40
ALU: ADDI x4, x0, 4
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
```

```
ALU: ADDI x4, x0, 8
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
ALU: ADDI x13, x13, 4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
ALU: ADDI x4, x0, 15
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
ALU: ADDI x13, x13, 8
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
ALU: ADDI x4, x0, 16
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
ALU: ADDI x13, x13, 12
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
ALU: ADDI x4, x0, 23
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
ALU: ADDI x13, x13, 16
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
ALU: ADDI x4, x0, 42
ALU: ADDI x13, x15, -24
ALU: NOP
ALU: NOP
ALU: ADDI x13, x13, 20
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x13)
ALU: ADDI x4, x0, 23
ALU: NOP
ALU: NOP
LSU: STORE x4, -28(x15)
ALU: ADDI x13, x14, -60
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
```

```
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 2416
pila_ok_18:
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x5, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x6, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x7, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x8, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x9, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x10, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x11, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x12, 0(x14)
LSU: LOAD x4, -28(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
```

```
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x13, x15, -24
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x13, 0(x14)
ALU: ADDI x14, x14, -4
ALU: ADDI x3, x1, 80
ALU: NOP
ALU: NOP
LSU: STORE x3, 0(x14) | BRU: JMP -4528
ALU: NOP
ret_call_19:
ALU: ADDI x14, x14, 8
ALU: NOP
ALU: NOP
LSU: LOAD x12, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x11, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x10, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x9, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x8, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x7, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x6, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x5, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x14)
ALU: ADDI x14, x14, 4
ALU: ADD x4, x0, x2
ALU: NOP
ALU: NOP
```

```
LSU: STORE x4, -32(x15)
ALU: ADDI x5, x0, 3 | LSU: LOAD x4, -32(x15)
ALU: NOP
ALU: ADDI x6, x0, 0
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 1
ALU: ADDI x9, x0, 0 | BRU: BGE x4, x0, 80
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x7, x7, 1
ALU: ADDI x9, x0, 1
div_izq_positivo_20:
BRU: BGE x5, x0, 48
ALU: SUB x5, x0, x5
ALU: ADDI x7, x7, 1
div_der_positivo_21:
ALU: NOP
BRU: BNE x5, x0, 32
BRU: JMP 96
div_divisor_valido_22:
div_bucle_23:
BRU: BGE x4, x5, 32
BRU: JMP 64
div_restar_24:
ALU: SUB x4, x4, x5
ALU: ADDI x6, x6, 1
BRU: JMP -64
div_fin_bucle_25:
BRU: BNE x7, x8, 32
ALU: SUB x6, x0, x6
div_fin_signo_cociente_26:
BRU: BNE x9, x8, 32
ALU: SUB x4, x0, x4
div_fin_signo_resto_27:
LSU: STORE x6, -36(x15)
ALU: ADDI x5, x0, 3 | LSU: LOAD x4, -32(x15)
ALU: NOP
ALU: ADDI x6, x0, 0
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 1
ALU: ADDI x9, x0, 0 | BRU: BGE x4, x0, 80
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x7, x7, 1
ALU: ADDI x9, x0, 1
div_izq_positivo_28:
BRU: BGE x5, x0, 48
ALU: SUB x5, x0, x5
ALU: ADDI x7, x7, 1
div_der_positivo_29:
ALU: NOP
BRU: BNE x5, x0, 32
BRU: JMP 96
div_divisor_valido_30:
```

```
div_bucle_31:
BRU: BGE x4, x5, 32
BRU: JMP 64
div_restar_32:
ALU: SUB x4, x4, x5
ALU: ADDI x6, x6, 1
BRU: JMP -64
div_fin_bucle_33:
BRU: BNE x7, x8, 32
ALU: SUB x6, x0, x6
div_fin_signo_cociente_34:
BRU: BNE x9, x8, 32
ALU: SUB x4, x0, x4
div_fin_signo_resto_35:
ALU: NOP
ALU: NOP
LSU: STORE x4, -40(x15)
ALU: ADD x14, x15, x0
BRU: JMP 16
fin_programa:
BRU: JMP 0
error_pila:
BRU: JMP 0
```

Representación hexadecimal legible.

```
000000000000000000000000000000E9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000008EEB
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000001EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000008EEB
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000008EEB
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000000EF1
0000000000004381000000000000000
00000000000000000000000000000029
000000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003BE00000000
000000000000000000000000000000EF1
```

```
0000000000000000000000000FF8EE9
00000000000000000000000000000049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FF3D200000000
00000000000000000000000000000049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FE3D200000000
0000000000000000003FF3D100002059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000003202A20000000000000000
00000000000000000000000000000049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FE3D200000000
0000000000000000003FE3D100003059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000002902A20000000000000000
0000000000000000003FF3D100000000
0000000000000000003FE3D500003069
00000000000000000000000000000000
00000000000000000000000000000079
00000000000000000000000000000089
00000000002002200000000000001099
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000001889
00000000001803200000000000000000
00000000000000000000000000006062
00000000000000000000000000001889
00000000000000000000000000000000
00000000004803400000000000000000
00000000000000000000000000096A5
00000000000000000000000000000000
00000000000000000000000000000000
00000000001005400000000000000000
00000000000000000000000000004771
0000000000000000000000000000144B
0000000000000000000000000000166A
00000000007FFB810000000000000000
00000000000104C50000000000000000
00000000000000000000000000007072
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000005771
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000277B
000000000000000000023F500000000
```

```
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000007DD1
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035100000000
00000000000000000000033D500000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000001002A50000000000000000
00000000000000000003FF3D100003059
00000000000000000000000000000000
00000000000000000000000000000069
00000000000000000000000000000079
00000000000200220000000000001089
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000001779
00000000001802A0000000000000000
00000000000000000000000000005052
00000000000000000000000000001779
00000000000000000000000000000000
00000000004802C0000000000000000
00000000000000000000000000008595
00000000000000000000000000000000
00000000000000000000000000000000
00000000001004C0000000000000000
00000000000000000000000000004661
00000000000000000000000000144B
00000000000000000000000000155A
00000000007FFB810000000000000000
00000000001043D0000000000000000
00000000000000000000000000006062
00000000000000003FE3D100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004661
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000006021
0000000000000D810000000000000000
00000000000000003FE3D100001059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000005441
00000000000000000000000000000000
00000000000000000000000000000000
00000000007FD601003FE3D200000000
00000000000000000000000000000000
00000000000000003FF3D100001059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
```

```
00000000000000000000000000005441
00000000000000000000000000000000
00000000000000000000000000000000
00000000007FCD01003FF3D200000000
00000000000000000000000000000000
00000000000000000000000000001049
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004021
00000000000000810000000000000000
0000000000000000000000000000FE1
000000000000000000003CD00000000
0000000000000000000013F500000000
00000000000000000000000000008FE9
0000000000000000000000000000003F1
000000000000000000000000000000D11
00000000000000000000000000000000
000000000000000000000000000000EF1
0000000000000000000000000FD8ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000008333B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000008333B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000008333B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
00000000000070810000000000000000
0000000000000000000000000FD8EE9
00000000000000000000000000004049
0000000000000000000000000FE8FD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
00000000000000000000000000008049
0000000000000000000000000FE8FD9
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000000000004DD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
0000000000000000000000000000F049
0000000000000000000000000000FE8FD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000008DD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
00000000000000000000000000010049
0000000000000000000000000000FE8FD9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000CDD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
00000000000000000000000000017049
0000000000000000000000000000FE8FD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000010DD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
00000000000000000000000000002A049
0000000000000000000000000000FE8FD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000014DD9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000035200000000
00000000000000000000000000017049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003F93D200000000
0000000000000000000000000000FC4ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
```

```
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
0000000000004B810000000000000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039600000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039A00000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039E00000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A200000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A600000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AA00000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AE00000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B200000000
0000000000000000003F93D100000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
000000000000000000000000FE8FD9
000000000000000000000000FFCEE9
```

```
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003B600000000
000000000000000000000000FFCEE9
00000000000000000000000000050139
00000000000000000000000000000000
00000000000000000000000000000000
00000000007F72810000038E00000000
00000000000000000000000000000000
000000000000000000000000000008EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003B100000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003AD00000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A900000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A500000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A100000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039D00000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039900000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039500000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039100000000
000000000000000000000000000004EE9
00000000000000000000000000002041
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003F83D200000000
0000000000000000003F83D100003059
00000000000000000000000000000000
00000000000000000000000000000069
00000000000000000000000000000079
```

```
00000000000000000000000000001089
00000000002802200000000000000099
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000001779
00000000000000000000000000001099
000000000001802A0000000000000000
00000000000000000000000000005052
00000000000000000000000000001779
00000000000000000000000000000000
000000000001002D0000000000000000
00000000000030100000000000000000
00000000000102A20000000000000000
00000000000020100000000000000000
00000000000000000000000000005442
00000000000000000000000000001669
00000000007FFE010000000000000000
000000000001043D0000000000000000
00000000000000000000000000006062
000000000001044D0000000000000000
00000000000000000000000000004042
00000000000000003F73DA00000000
00000000000000003F83D100003059
00000000000000000000000000000000
00000000000000000000000000000069
00000000000000000000000000000079
00000000000000000000000000001089
00000000002802200000000000000099
00000000000000000000000000000000
00000000000000000000000000004042
00000000000000000000000000001779
00000000000000000000000000001099
000000000001802A0000000000000000
00000000000000000000000000005052
00000000000000000000000000001779
00000000000000000000000000000000
000000000001002D0000000000000000
00000000000030100000000000000000
00000000000102A20000000000000000
00000000000020100000000000000000
00000000000000000000000000005442
00000000000000000000000000001669
00000000007FFE010000000000000000
000000000001043D0000000000000000
00000000000000000000000000006062
000000000001044D0000000000000000
00000000000000000000000000004042
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000003F63D200000000
00000000000000000000000000000FE1
00000000000000810000000000000000
00000000000000010000000000000000
00000000000000010000000000000000
```

Traza de ejecución. La traza final verifica los valores de las variables principales en memoria: `objetivo=23`, `posicion=4`, `fila=1` y `columna=1`. También muestra los saltos tomados durante los ciclos y el bucle terminal del programa.

```
BRANCH PC=00000160 -> 000009d0
BRANCH PC=00000b40 -> 00000b60
STORE MEM[0000ffe8] <= 00000004 (4)
STORE MEM[0000ffec] <= 00000008 (8)
STORE MEM[0000fff0] <= 0000000f (15)
STORE MEM[0000fff4] <= 00000010 (16)
STORE MEM[0000fff8] <= 00000017 (23)
STORE MEM[0000fffc] <= 0000002a (42)
STORE MEM[0000ffe4] <= 00000017 (23)
BRANCH PC=00000fe0 -> 00001000
STORE MEM[0000ffd4] <= 00000017 (23)
STORE MEM[0000ffd0] <= 00000000 (0)
STORE MEM[0000ffcc] <= 00000000 (0)
STORE MEM[0000ffc8] <= 00000000 (0)
STORE MEM[0000ffc4] <= 00000000 (0)
STORE MEM[0000ffc0] <= 00000000 (0)
STORE MEM[0000ffbc] <= 00000000 (0)
STORE MEM[0000ffb8] <= 00000000 (0)
STORE MEM[0000ffb4] <= 00000000 (0)
STORE MEM[0000ffb0] <= 00000017 (23)
STORE MEM[0000ffac] <= 0000ffe8 (65512)
STORE MEM[0000ffa8] <= 00001340 (4928)
BRANCH PC=00001320 -> 00000170
STORE MEM[0000ffa4] <= 00010000 (65536)
STORE MEM[0000ffa0] <= 00000000 (0)
STORE MEM[0000ff9c] <= 00000000 (0)
STORE MEM[0000ff9c] <= 00000000 (0)
BRANCH PC=00000390 -> 000003d0
BRANCH PC=000003d0 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000410 -> 000004a0
BRANCH PC=000004a0 -> 000004c0
BRANCH PC=000005c0 -> 000007c0
STORE MEM[0000ff9c] <= 00000001 (1)
BRANCH PC=00000830 -> 000002f0
BRANCH PC=00000390 -> 000003d0
BRANCH PC=000003d0 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000410 -> 000004a0
BRANCH PC=000004a0 -> 000004c0
BRANCH PC=000005c0 -> 000007c0
STORE MEM[0000ff9c] <= 00000002 (2)
BRANCH PC=00000830 -> 000002f0
BRANCH PC=00000390 -> 000003d0
BRANCH PC=000003d0 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000490 -> 00000400
BRANCH PC=00000410 -> 000004a0
BRANCH PC=000004a0 -> 000004c0
BRANCH PC=000005c0 -> 000007c0
STORE MEM[0000ff9c] <= 00000003 (3)
BRANCH PC=00000830 -> 000002f0
BRANCH PC=00000330 -> 00000850
STORE MEM[0000ffa0] <= 00000001 (1)
BRANCH PC=000008c0 -> 00000260
STORE MEM[0000ff9c] <= 00000000 (0)
```

```
  BRANCH PC=00000390 -> 000003d0
  BRANCH PC=000003d0 -> 00000400
  BRANCH PC=00000490 -> 00000400
  BRANCH PC=00000490 -> 00000400
  BRANCH PC=00000410 -> 000004a0
  BRANCH PC=000004a0 -> 000004c0
  BRANCH PC=000005c0 -> 000007c0
  STORE MEM[0000ff9c] <= 00000001 (1)
  BRANCH PC=00000830 -> 000002f0
  BRANCH PC=00000390 -> 000003d0
  BRANCH PC=000003d0 -> 00000400
  BRANCH PC=00000490 -> 00000400
  BRANCH PC=00000490 -> 00000400
  BRANCH PC=00000410 -> 000004a0
  BRANCH PC=000004a0 -> 000004c0
  BRANCH PC=00000610 -> 00000650
  BRANCH PC=00000650 -> 00000680
  BRANCH PC=00000710 -> 00000680
  BRANCH PC=00000710 -> 00000680
  BRANCH PC=00000690 -> 00000720
  BRANCH PC=00000720 -> 00000740
  BRANCH PC=000007b0 -> 00000960
  STORE MEM[0000ffe0] <= 00000004 (4)
  BRANCH PC=00001620 -> 00001670
  BRANCH PC=00001670 -> 000016a0
  BRANCH PC=000016b0 -> 000016d0
  BRANCH PC=000016d0 -> 000016f0
  BRANCH PC=00001710 -> 000016d0
  BRANCH PC=000016e0 -> 00001720
  BRANCH PC=00001720 -> 00001740
  BRANCH PC=00001740 -> 00001760
  STORE MEM[0000ffdc] <= 00000001 (1)
  BRANCH PC=000017c0 -> 00001810
  BRANCH PC=00001810 -> 00001840
  BRANCH PC=00001850 -> 00001870
  BRANCH PC=00001870 -> 00001890
  BRANCH PC=000018b0 -> 00001870
  BRANCH PC=00001880 -> 000018c0
  BRANCH PC=000018c0 -> 000018e0
  BRANCH PC=000018e0 -> 00001900
  STORE MEM[0000ffd8] <= 00000001 (1)
  BRANCH PC=00001940 -> 00001950
  BRANCH PC=00001950 -> 00001950
SELF_LOOP detectado en PC=0x00001950
PASS: fin_programa alcanzado en 0x00001950
x0 = 0x00000000 (0)
x1 = 0x00000000 (0)
x2 = 0x00000004 (4)
x3 = 0x00010000 (65536)
x4 = 0x00000001 (1)
x5 = 0x00000003 (3)
x6 = 0x00000001 (1)
x7 = 0x00000000 (0)
x8 = 0x00000001 (1)
x9 = 0x00000000 (0)
x10 = 0x00000000 (0)
x11 = 0x00000000 (0)
x12 = 0x00000000 (0)
x13 = 0x00001340 (4928)
x14 = 0x00010000 (65536)
x15 = 0x00010000 (65536)
MEM[0000ff9c] = 00000001 (1)
MEM[0000ffa0] = 00000001 (1)
```

```
MEM[0000ffa4] = 00010000 (65536)
MEM[0000ffa8] = 00001340 (4928)
MEM[0000ffac] = 0000ffe8 (65512)
MEM[0000ffb0] = 00000017 (23)
MEM[0000ffd4] = 00000017 (23)
MEM[0000ffd8] = 00000001 (1)
MEM[0000ffdc] = 00000001 (1)
MEM[0000ffe0] = 00000004 (4)
MEM[0000ffe4] = 00000017 (23)
MEM[0000ffe8] = 00000004 (4)
MEM[0000ffec] = 00000008 (8)
MEM[0000fff0] = 0000000f (15)
MEM[0000fff4] = 00000010 (16)
MEM[0000fff8] = 00000017 (23)
MEM[0000fffc] = 0000002a (42)
```

8.4.3. Ejercicio 3: máximo común divisor

Como programa propuesto se utilizó el cálculo del máximo común divisor de 48 y 18. El programa emplea funciones con parámetros y retorno, incluyendo una función auxiliar de valor absoluto, llamadas entre funciones y estructuras de control para implementar el algoritmo. La ejecución produjo `MCD(48,18)=6` y alcanzó `fin_programa` en `0x00002000`.

Código fuente completo.

```
defina entero absoluto(entero n) {
    si (n < 0) {
        devuelva 0 - n :)
    } :)

    devuelva n :)
} :)

defina entero mcd(entero a, entero b) {
    entero residuo = 0 :)

    mientras (b != 0) {
        residuo = a % b :)

        si (residuo < 0) {
            residuo = 0 - residuo :)
        } :)

        a = b :)
        b = residuo :)
    } :)

    devuelva a :)
} :)

defina vacio principal() {
    entero numero1 = 48 :)
    entero numero2 = 18 :)

    entero positivo1 = absoluto(numero1) :)
    entero positivo2 = absoluto(numero2) :)

    entero resultado = mcd(positivo1, positivo2) :)
} :)
```

Resumen de compilación.

```
[FASES 1-2] Analisis lexico, sintactico y construccion del AST...
Programa sintacticamente correcto.
[FASE 3] Analisis semantico finalizado sin errores.
[FASE 4] Generando ensamblador y calendarizando dependencias...
[FASE 5] Resolviendo etiquetas y offsets...
Fase 5: 514 bundles resueltos correctamente
[FASE 5.5] Empaquetando operaciones independientes en bundles VLIW...
Empaquetado VLIW: 514 bundles conservados
  ALU + LSU: 4
  ALU + BRU: 1
```

```
  LSU + BRU: 4
[FASE 6] Generando codigo maquina...
Archivo binario generado: ejercicio3_mcd.bin
Bundles generados: 514
Bytes de instrucciones: 8224
```

Ensamblador intermedio generado.

```
ALU: ADDI x14, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 1
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: SLLI x14, x14, 8
ALU: NOP
ALU: NOP
ALU: ADDI x14, x14, 0
ALU: NOP
ALU: NOP
ALU: ADD x15, x14, x0
BRU: JMP 1808
func_absoluto:
ALU: ADDI x2, x0, 0
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x15, 0(x14)
ALU: ADD x15, x14, x0
ALU: NOP
ALU: NOP
ALU: ADDI x5, x0, 0 | LSU: LOAD x4, 8(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BGE x4, x5, 160
ALU: ADDI x4, x0, 0
LSU: LOAD x5, 8(x15)
ALU: NOP
ALU: NOP
ALU: SUB x4, x4, x5
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 96
fin_if_0:
```

```
LSU: LOAD x4, 8(x15)
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 16
ret_absoluto:
ALU: ADD x14, x15, x0
LSU: LOAD x3, 0(x15)
LSU: LOAD x13, 4(x15)
ALU: ADDI x14, x15, 8
ALU: ADD x15, x3, x0
ALU: ADD x1, x13, x0
ALU: NOP
func_mcd:
ALU: ADDI x2, x0, 0
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x15, 0(x14)
ALU: ADD x15, x14, x0
ALU: ADDI x14, x14, -4
ALU: ADDI x4, x0, 0
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
inicio_while_1:
ALU: ADDI x5, x0, 0 | LSU: LOAD x4, 12(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BEQ x4, x5, 816
LSU: LOAD x4, 8(x15)
ALU: ADDI x6, x0, 0 | LSU: LOAD x5, 12(x15)
ALU: NOP
ALU: ADDI x7, x0, 0
ALU: ADDI x8, x0, 1
ALU: ADDI x9, x0, 0 | BRU: BGE x4, x0, 80
ALU: NOP
ALU: SUB x4, x0, x4
ALU: ADDI x7, x7, 1
ALU: ADDI x9, x0, 1
div_izq_positivo_3:
BRU: BGE x5, x0, 48
ALU: SUB x5, x0, x5
ALU: ADDI x7, x7, 1
div_der_positivo_4:
ALU: NOP
BRU: BNE x5, x0, 32
BRU: JMP 96
div_divisor_valido_5:
div_bucle_6:
BRU: BGE x4, x5, 32
BRU: JMP 64
div_restar_7:
```

```
ALU: SUB x4, x4, x5
ALU: ADDI x6, x6, 1
BRU: JMP -64
div_fin_bucle_8:
BRU: BNE x7, x8, 32
ALU: SUB x6, x0, x6
div_fin_signo_cociente_9:
BRU: BNE x9, x8, 32
ALU: SUB x4, x0, x4
div_fin_signo_resto_10:
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
ALU: ADDI x5, x0, 0 | LSU: LOAD x4, -4(x15)
ALU: NOP
ALU: NOP
ALU: NOP
BRU: BGE x4, x5, 144
ALU: ADDI x4, x0, 0
LSU: LOAD x5, -4(x15)
ALU: NOP
ALU: NOP
ALU: SUB x4, x4, x5
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
fin_if_11:
LSU: LOAD x4, 12(x15)
ALU: NOP
ALU: NOP
LSU: STORE x4, 8(x15)
LSU: LOAD x4, -4(x15)
ALU: NOP
ALU: NOP
LSU: STORE x4, 12(x15) | BRU: JMP -848
ALU: NOP
fin_while_2:
LSU: LOAD x4, 8(x15)
ALU: NOP
ALU: NOP
ALU: ADD x2, x0, x4
BRU: JMP 16
ret_mcd:
ALU: ADD x14, x15, x0
LSU: LOAD x3, 0(x15)
LSU: LOAD x13, 4(x15)
ALU: ADDI x14, x15, 8
ALU: ADD x15, x3, x0
ALU: ADD x1, x13, x0
ALU: NOP
principal:
ALU: ADD x15, x14, x0
ALU: ADDI x13, x14, -20
ALU: ADDI x3, x0, 0
```

```
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 5664
pila_ok_12:
ALU: ADDI x14, x14, -20
ALU: ADDI x4, x0, 48
ALU: NOP
ALU: NOP
LSU: STORE x4, -4(x15)
ALU: ADDI x4, x0, 18
ALU: NOP
ALU: NOP
LSU: STORE x4, -8(x15)
ALU: ADDI x13, x14, -48
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
```

```
BRU: BGE x13, x3, 32
BRU: JMP 5136
pila_ok_13:
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x5, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x6, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x7, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x8, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x9, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x10, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x11, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x12, 0(x14)
LSU: LOAD x4, -4(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: ADDI x3, x1, 80
ALU: NOP
ALU: NOP
LSU: STORE x3, 0(x14) | BRU: JMP -3440
ALU: NOP
ret_call_14:
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
```

```
LSU: LOAD x12, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x11, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x10, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x9, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x8, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x7, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x6, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x5, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x14)
ALU: ADDI x14, x14, 4
ALU: ADD x4, x0, x2
ALU: NOP
ALU: NOP
LSU: STORE x4, -12(x15)
ALU: ADDI x13, x14, -48
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
```

```
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 3344
pila_ok_15:
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x5, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x6, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x7, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x8, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x9, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x10, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x11, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x12, 0(x14)
LSU: LOAD x4, -8(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: ADDI x3, x1, 80
ALU: NOP
ALU: NOP
```

```
LSU: STORE x3, 0(x14) | BRU: JMP -5232
ALU: NOP
ret_call_16:
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x12, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x11, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x10, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x9, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x8, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x7, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x6, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x5, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x14)
ALU: ADDI x14, x14, 4
ALU: ADD x4, x0, x2
ALU: NOP
ALU: NOP
LSU: STORE x4, -16(x15)
ALU: ADDI x13, x14, -56
ALU: ADDI x3, x0, 0
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
```

```
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 128
ALU: NOP
ALU: NOP
ALU: SLLI x3, x3, 8
ALU: NOP
ALU: NOP
ALU: ADDI x3, x3, 0
ALU: NOP
ALU: NOP
BRU: BGE x13, x3, 32
BRU: JMP 1552
pila_ok_17:
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x5, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x6, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x7, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x8, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x9, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x10, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x11, 0(x14)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x12, 0(x14)
LSU: LOAD x4, -16(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
```

```
ALU: NOP
LSU: STORE x4, 0(x14)
LSU: LOAD x4, -12(x15)
ALU: ADDI x14, x14, -4
ALU: NOP
ALU: NOP
LSU: STORE x4, 0(x14)
ALU: ADDI x14, x14, -4
ALU: ADDI x3, x1, 80
ALU: NOP
ALU: NOP
LSU: STORE x3, 0(x14) | BRU: JMP -6560
ALU: NOP
ret_call_18:
ALU: ADDI x14, x14, 8
ALU: NOP
ALU: NOP
LSU: LOAD x12, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x11, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x10, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x9, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x8, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x7, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x6, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x5, 0(x14)
ALU: ADDI x14, x14, 4
ALU: NOP
ALU: NOP
LSU: LOAD x4, 0(x14)
ALU: ADDI x14, x14, 4
ALU: ADD x4, x0, x2
ALU: NOP
ALU: NOP
```

```
LSU: STORE x4, -20(x15)
ALU: ADD x14, x15, x0
BRU: JMP 16
fin_programa:
BRU: JMP 0
error_pila:
BRU: JMP 0
```

Representación hexadecimal legible.

```
000000000000000000000000000000E9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000001EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000EEB
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000EF1
0000000000003881000000000000000
00000000000000000000000000000029
00000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003BE00000000
00000000000000000000000000000EF1
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000023D100000059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000502A20000000000000000
00000000000000000000000000000049
00000000000000000023D500000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000005442
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004021
00000000000030100000000000000000
```

```
00000000000000000000023D100000000
00000000000000000000000000000000000
00000000000000000000000000000000000
00000000000000000000000000000004021
00000000000000810000000000000000000
00000000000000000000000000000FE1
00000000000000000000003CD00000000
00000000000000000000013F500000000
00000000000000000000000000008FE9
00000000000000000000000000003F1
00000000000000000000000000000D11
00000000000000000000000000000000000
00000000000000000000000000000029
00000000000000000000000000FFCEE9
00000000000000000000000000000000000
00000000000000000000000000000000000
00000000000000000000003BE00000000
00000000000000000000000000000EF1
00000000000000000000000000FFCEE9
00000000000000000000000000000049
00000000000000000000000000000000000
00000000000000000000000000000000000
00000000000000000003FF3D200000000
00000000000000000000033D100000059
00000000000000000000000000000000000
00000000000000000000000000000000000
00000000000000000000000000000000000
00000000001982A40000000000000000000
00000000000000000000023D100000000
00000000000000000000033D500000069
00000000000000000000000000000000000
00000000000000000000000000000000079
00000000000000000000000000000001089
00000000000280220000000000000099
00000000000000000000000000000000000
00000000000000000000000000000004042
00000000000000000000000000000001779
00000000000000000000000000000001099
00000000001802A0000000000000000000
00000000000000000000000000000005052
00000000000000000000000000000001779
00000000000000000000000000000000000
00000000001002D0000000000000000000
00000000000030100000000000000000000
00000000000102A20000000000000000000
00000000000020100000000000000000000
00000000000000000000000000000005442
00000000000000000000000000000001669
00000000007FFE010000000000000000000
00000000001043D0000000000000000000
00000000000000000000000000000006062
00000000001044D0000000000000000000
00000000000000000000000000000004042
00000000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000003FF3D200000000
00000000000000003FF3D100000059
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000482A20000000000000000
00000000000000000000000000000049
00000000000000003FF3D500000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000005442
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000003FF3D200000000
00000000000000000000033D100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000023D200000000
00000000000000003FF3D100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000007FE581000033D200000000
00000000000000000000000000000000
00000000000000000000023D100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000004021
00000000000000810000000000000000
00000000000000000000000000000FE1
00000000000000000000003CD00000000
00000000000000000000013F500000000
00000000000000000000000000008FE9
00000000000000000000000000003F1
00000000000000000000000000000D11
00000000000000000000000000000000
00000000000000000000000000000EF1
00000000000000000000000000FECED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000833B
```

```
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
000000000000B1010000000000000000
0000000000000000000000000000FECEE9
00000000000000000000000000030049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FF3D200000000
00000000000000000000000000012049
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FE3D200000000
00000000000000000000000000FD0ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
000000000000A0810000000000000000
00000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
00000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039600000000
00000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039A00000000
00000000000000000000000000FFCEE9
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000039E00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A200000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AA00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AE00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B200000000
00000000000000000003FF3D100000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
0000000000000000000000000FFCEE9
00000000000000000000000000050139
00000000000000000000000000000000
00000000000000000000000000000000
00000000007F94810000038E00000000
00000000000000000000000000000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AD00000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A500000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A100000000
00000000000000000000000000004EE9
```

```
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039D00000000
0000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039900000000
0000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039500000000
0000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039100000000
0000000000000000000000000000004EE9
00000000000000000000000000002041
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FD3D200000000
000000000000000000000000FD0ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
00000000000068810000000000000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039600000000
000000000000000000000000FFCEE9
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000039A00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039E00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A200000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AA00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AE00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B200000000
00000000000000000003FE3D100000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
0000000000000000000000000FFCEE9
00000000000000000000000000050139
00000000000000000000000000000000
00000000000000000000000000000000
00000000007F5C810000038E00000000
00000000000000000000000000000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AD00000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A500000000
00000000000000000000000000004EE9
```

```
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000003A100000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039D00000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039900000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039500000000
000000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039100000000
000000000000000000000000000004EE9
00000000000000000000000000002041
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000003FC3D200000000
000000000000000000000000000FC8ED9
00000000000000000000000000000039
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000080339
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000000000000000000833B
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000339
00000000000000000000000000000000
00000000000000000000000000000000
00000000000101EA0000000000000000
00000000000030810000000000000000
000000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
0000000000000000000039200000000
000000000000000000000000000FFCEE9
00000000000000000000000000000000
```

```
00000000000000000000000000000000
00000000000000000000039600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039A00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039E00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A200000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A600000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AA00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AE00000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B200000000
00000000000000000003FC3D100000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
00000000000000000003FD3D100000000
0000000000000000000000000FFCEE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039200000000
0000000000000000000000000FFCEE9
00000000000000000000000000050139
00000000000000000000000000000000
00000000000000000000000000000000
00000000007F33010000038E00000000
00000000000000000000000000000000
00000000000000000000000000008EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003B100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003AD00000000
```

```
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A500000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000003A100000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039D00000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039900000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039500000000
00000000000000000000000000004EE9
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000039100000000
00000000000000000000000000004EE9
00000000000000000000000000002041
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000003FB3D200000000
00000000000000000000000000000FE1
00000000000000081000000000000000
00000000000000001000000000000000
00000000000000001000000000000000
```

Traza de ejecución. Durante la ejecución se observan los valores intermedios del algoritmo hasta llegar a 6. El valor final queda en `x2=6`, y la traza termina con la detección del bucle de `fin_programa`.

```
BRANCH PC=00000160 -> 00000870
BRANCH PC=000009e0 -> 00000a00
STORE MEM[0000fffc] <= 00000030 (48)
STORE MEM[0000fff8] <= 00000012 (18)
BRANCH PC=00000bf0 -> 00000c10
STORE MEM[0000ffe8] <= 00000012 (18)
STORE MEM[0000ffe4] <= 00000000 (0)
STORE MEM[0000ffe0] <= 00000000 (0)
STORE MEM[0000ffdc] <= 00000000 (0)
STORE MEM[0000ffd8] <= 00000000 (0)
STORE MEM[0000ffd4] <= 00000000 (0)
STORE MEM[0000ffd0] <= 00000000 (0)
STORE MEM[0000ffcc] <= 00000000 (0)
STORE MEM[0000ffc8] <= 00000000 (0)
```

```
STORE MEM[0000ffc4] <= 00000030 (48)
STORE MEM[0000ffc0] <= 00000f00 (3840)
BRANCH PC=00000ee0 -> 00000170
STORE MEM[0000ffbc] <= 00010000 (65536)
BRANCH PC=00000230 -> 000002d0
BRANCH PC=00000310 -> 00000320
STORE MEM[0000fff4] <= 00000030 (48)
BRANCH PC=000012f0 -> 00001310
STORE MEM[0000ffe8] <= 00000030 (48)
STORE MEM[0000ffe4] <= 00000000 (0)
STORE MEM[0000ffe0] <= 00000000 (0)
STORE MEM[0000ffdc] <= 00000000 (0)
STORE MEM[0000ffd8] <= 00000000 (0)
STORE MEM[0000ffd4] <= 00000000 (0)
STORE MEM[0000ffd0] <= 00000000 (0)
STORE MEM[0000ffcc] <= 00000000 (0)
STORE MEM[0000ffc8] <= 00000000 (0)
STORE MEM[0000ffc4] <= 00000012 (18)
STORE MEM[0000ffc0] <= 00001600 (5632)
BRANCH PC=000015e0 -> 00000170
STORE MEM[0000ffbc] <= 00010000 (65536)
BRANCH PC=00000230 -> 000002d0
BRANCH PC=00000310 -> 00000320
STORE MEM[0000fff0] <= 00000012 (18)
BRANCH PC=000019f0 -> 00001a10
STORE MEM[0000ffe8] <= 00000012 (18)
STORE MEM[0000ffe4] <= 00000000 (0)
STORE MEM[0000ffe0] <= 00000000 (0)
STORE MEM[0000ffdc] <= 00000000 (0)
STORE MEM[0000ffd8] <= 00000000 (0)
STORE MEM[0000ffd4] <= 00000000 (0)
STORE MEM[0000ffd0] <= 00000000 (0)
STORE MEM[0000ffcc] <= 00000000 (0)
STORE MEM[0000ffc8] <= 00000000 (0)
STORE MEM[0000ffc4] <= 00000012 (18)
STORE MEM[0000ffc0] <= 00000030 (48)
STORE MEM[0000ffbc] <= 00001d50 (7504)
BRANCH PC=00001d30 -> 00000390
STORE MEM[0000ffb8] <= 00010000 (65536)
STORE MEM[0000ffb4] <= 00000000 (0)
BRANCH PC=000004e0 -> 00000530
BRANCH PC=00000530 -> 00000560
BRANCH PC=00000570 -> 00000590
BRANCH PC=00000590 -> 000005b0
BRANCH PC=000005d0 -> 00000590
BRANCH PC=00000590 -> 000005b0
BRANCH PC=000005d0 -> 00000590
BRANCH PC=000005a0 -> 000005e0
BRANCH PC=000005e0 -> 00000600
BRANCH PC=00000600 -> 00000620
STORE MEM[0000ffb4] <= 0000000c (12)
BRANCH PC=00000690 -> 00000720
STORE MEM[0000ffc0] <= 00000012 (18)
STORE MEM[0000ffc4] <= 0000000c (12)
BRANCH PC=00000790 -> 00000440
BRANCH PC=000004e0 -> 00000530
BRANCH PC=00000530 -> 00000560
BRANCH PC=00000570 -> 00000590
BRANCH PC=00000590 -> 000005b0
BRANCH PC=000005d0 -> 00000590
BRANCH PC=000005a0 -> 000005e0
BRANCH PC=000005e0 -> 00000600
BRANCH PC=00000600 -> 00000620
```

```
  STORE MEM[0000ffb4] <= 00000006 (6)
  BRANCH PC=00000690 -> 00000720
  STORE MEM[0000ffc0] <= 0000000c (12)
  STORE MEM[0000ffc4] <= 00000006 (6)
  BRANCH PC=00000790 -> 00000440
  BRANCH PC=000004e0 -> 00000530
  BRANCH PC=00000530 -> 00000560
  BRANCH PC=00000570 -> 00000590
  BRANCH PC=00000590 -> 000005b0
  BRANCH PC=000005d0 -> 00000590
  BRANCH PC=00000590 -> 000005b0
  BRANCH PC=000005d0 -> 00000590
  BRANCH PC=000005a0 -> 000005e0
  BRANCH PC=000005e0 -> 00000600
  BRANCH PC=00000600 -> 00000620
  STORE MEM[0000ffb4] <= 00000000 (0)
  BRANCH PC=00000690 -> 00000720
  STORE MEM[0000ffc0] <= 00000006 (6)
  STORE MEM[0000ffc4] <= 00000000 (0)
  BRANCH PC=00000790 -> 00000440
  BRANCH PC=00000480 -> 000007b0
  BRANCH PC=000007f0 -> 00000800
  STORE MEM[0000ffec] <= 00000006 (6)
  BRANCH PC=00001ff0 -> 00002000
  BRANCH PC=00002000 -> 00002000
SELF_LOOP detectado en PC=0x00002000
PASS: fin_programa alcanzado en 0x00002000
x0 = 0x00000000 (0)
x1 = 0x00000000 (0)
x2 = 0x00000006 (6)
x3 = 0x00010000 (65536)
x4 = 0x00000006 (6)
x5 = 0x00000000 (0)
x6 = 0x00000000 (0)
x7 = 0x00000000 (0)
x8 = 0x00000000 (0)
x9 = 0x00000000 (0)
x10 = 0x00000000 (0)
x11 = 0x00000000 (0)
x12 = 0x00000000 (0)
x13 = 0x00001d50 (7504)
x14 = 0x00010000 (65536)
x15 = 0x00010000 (65536)
MEM[0000ffb8] = 00010000 (65536)
MEM[0000ffbc] = 00001d50 (7504)
MEM[0000ffc0] = 00000006 (6)
MEM[0000ffe8] = 00000012 (18)
MEM[0000ffec] = 00000006 (6)
MEM[0000fff0] = 00000012 (18)
MEM[0000fff4] = 00000030 (48)
MEM[0000fff8] = 00000012 (18)
MEM[0000fffc] = 00000030 (48)
```

Resultado de la integración. Los tres programas fueron compilados por la versión final del compilador y ejecutados en el procesador VLIW de CE4301. En los tres casos se alcanzó el PC terminal esperado y los valores observados coincidieron con el resultado semántico esperado. Esta prueba valida de extremo a extremo el flujo `fuente .e` → `ensamblador` → `HEX VLIW` → `ejecución RTL`.

8.5. Manejo de Casos Especiales

8.5.1. Saltos anidados

Cada estructura de control genera etiquetas únicas. Un contador interno evita colisiones entre etiquetas pertenecientes a distintos `si`, `mientras` o `para`. La sentencia `rompa` utiliza la etiqueta de salida del ciclo más interno activo. Al resolver las referencias se calcula la posición definitiva de cada etiqueta después de la calendarización.

8.5.2. Llamadas a función y retorno de valores

El compilador utiliza `x14` como SP, `x15` como FP y `x2` para el valor retornado. Los argumentos se colocan en la pila y el callee crea un nuevo marco. El retorno no depende de instrucciones `CALL/RET`, ya que la ISA no las posee; se implementa mediante `JMP` para la llamada y una escritura a `x1` para regresar.

8.5.3. Variables con el mismo nombre en distintos ámbitos

Cada símbolo conserva el identificador del ámbito donde fue declarado. La resolución busca primero en el ámbito actual y luego asciende por la jerarquía. Por ello, una variable local puede ocultar temporalmente una variable externa con el mismo nombre sin alterar la entrada correspondiente al ámbito superior.

8.5.4. Asignación de memoria local y global

Las variables globales utilizan direcciones absolutas dentro de `0x0000-0x7FFF`. Los parámetros y variables locales se ubican en el marco de la función y se acceden mediante desplazamientos respecto a FP. Las listas y matrices reservan espacio contiguo para todos sus elementos; cuando se pasan como parámetros se utiliza una dirección de 4 bytes.

Las evidencias de la Sección 8.4 corresponden a la versión final validada del compilador y a ejecuciones reales sobre el procesador VLIW.